East West University
Department of Computer Science and Engineering
Spring '23

Course Code: CSE366

Course Title: Artificial Intelligence

Section: 02

Project Title: Read and print the "Predicting Passenger Survival on the Titanic Collection Dataset" in the console.

Submitted by:
Jannatul Ferdous Salma
2020-2-60-005

Submitted to:
Md Ashraful Islam
Lecturer
Department of CSE

Date of Submission: 04 May 2023

Abstract:

I'll use the "Predicting Passenger Survival on the Titanic Collection Dataset" for this project, which I Collected from Kaggle. The Titanic's dataset in issue comprises data on the passengers of the Titanic, including whether or not they survived the disaster. The dataset, which includes 418 rows and 12 columns, is used for binary classification tasks, in which the purpose is to predict, based on a passenger's attributes such as age, gender, ticket class, and others, whether a particular passenger survived.

In this project, we aim to build a binary classification model using machine learning algorithms that can accurately predict whether a passenger survived or not based on the given features. We will begin by exploring the dataset and performing data cleaning and feature engineering to prepare the data for analysis. Then, we will compare the performance of various classification algorithms, including Bayesian and Naive Bayesian Classifier, decision trees, and Perceptron, to determine the best model for this dataset.

We chose this dataset because it is a well-known and frequently used dataset in the machine learning community for binary classification tasks. Additionally, the dataset provides an interesting and tragic historical event that adds context and meaning to the analysis and results.

Data description:

The Titanic dataset is a well-known dataset that is frequently used in the machine-learning field for binary classification problems. This is mainly because of its historical observations on the histories of Titanic passengers and the range of variables it provides.

This data set includes information on 891 Titanic passengers as well as 418 others that will be analyzed. The data set's rows each correspond to a passenger and contain details like the number of passengers, the travel plan, and the passenger's classification as a survivor.

The features or columns in the dataset include:

- PassengerId: A unique identifier for each passenger
- Survived: Whether or not the passenger survived the disaster (0 = No, 1 = Yes)
- Pclass: The passenger's class (1 = 1st, 2 = 2nd, 3 = 3rd)
- Name: The passenger's name
- Sex: The passenger's sex (male or female)
- Age: The passenger's age in years
- SibSp: The number of siblings or spouses the passenger had on board
- Parch: The number of parents or children the passenger had on board
- Ticket: The passenger's ticket number
- Fare: The fare the passenger paid for their ticket
- Cabin: The cabin number the passenger was assigned to
- Embarked: The port where the passenger embarked (C = Cherbourg, Q = Queenstown, S = Southampton)

The data cleaning and preparation stage will need to deal with particular missing values for some attributes in the data collection. Additionally, existing features can be updated and modified to provide new features that might be more helpful in predicting passenger survival.

The goal of using this dataset is to build a binary classification model that can accurately predict whether a passenger survived or not based on the given features. This dataset has been used for a wide variety of machine learning projects and provides an excellent opportunity to explore and experiment with different classification algorithms and techniques.

Implementation:

I used Google Colab to develop the code to read and output the dataset. At first, I import the necessary package to mount Google Drive in Colab. The drive has been mounted using the mount.drive() method.

```
from google.colab import drive
drive.mount('/content/drive')
```

Figure 01: Mounting Google Drive in Colab

This line mounts in Google Drive in Colab and sets the mount point to the directory `/content/drive`. Once this line is run, I will be prompted to follow a link and enter an authorization code to grant Colab permission to access my Google Drive account.

Overall, these two lines of code are used in Colab to mount my Google Drive account and allow me to access files and folders that are stored there. This is typically done when I have data files that are stored in my Google Drive account that I want to use in my Colab notebook.

```
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/tested.csv')
df
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
413	1305	0	3	Spector, Mr. Woolf	male	NaN	0	0	A.5. 3236	8.0500	NaN	S
414	1306	1	1	Oliva y Ocana, Dona. Fermina	female	39.0	0	0	PC 17758	108.9000	C105	C
415	1307	0	3	Saether, Mr. Simon Sivertsen	male	38.5	0	0	SOTON/O.Q. 3101262	7.2500	NaN	S
416	1308	0	3	Ware, Mr. Frederick	male	NaN	0	0	359309	8.0500	NaN	S
417	1309	0	3	Peter, Master. Michael J	male	NaN	1	1	2668	22.3583	NaN	C

418 rows × 12 columns

Figure 02: Reading and Printing the Dataset

The initial line imports the Pandas library and provides it a name such as `pd` so that it may be easier referred to in code. Then, in the second line, data is stored in a Pandas dataframe with the name `df` after being read from a CSV file located at `/content/tested.csv`. The CSV file is read into a dataframe by using the `pd.read_csv()` method, with the file path '/content/tested.csv' identifying the file's location. The final line only shows the `df` dataframe contents in the output. I should see the contents of the `tested.csv` file presented in a tabular form when I run this line.

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  418 non-null    int64
 1   Survived     418 non-null    int64
 2   Pclass       418 non-null    int64
 3   Name         418 non-null    object
 4   Sex          418 non-null    object
 5   Age          332 non-null    float64
 6   SibSp        418 non-null    int64
 7   Parch        418 non-null    int64
 8   Ticket       418 non-null    object
 9   Fare         417 non-null    float64
 10  Cabin        91 non-null     object
 11  Embarked     418 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 39.3+ KB
None
```

Figure 03: Non-null Value Counts

A short overview of a Pandas dataframe, including the data types of each column and the number of non-null values, can be obtained using the `df.info()` method. After reading the tested.csv file, if I run `print(df.info ()) `, it will show the following information:The total number of rows in the dataframe, the number of non-null values in each column and data type of each column.

```
] df.duplicated()

0      False
1      False
2      False
3      False
4      False
       ...
413    False
414    False
415    False
416    False
417    False
Length: 418, dtype: bool
```

Figure 04: Checking Duplicate Value

To find duplicate rows in a Pandas dataframe and produce boolean values for them, use the `df.duplicated()` method. A dataframe's `df.duplicated()` function returns a boolean series with True denoting that a row is a duplicate of a previous row and False denoting that the row is not a duplicate.

```
df.isnull().sum()

PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

Figure 05: Null Value Count

This `df.isnull().sum()` method is used to identify the number of missing values (NaN) in each column of a Pandas dataframe. It returns a series object where each column name is the index and the corresponding value is the count of missing values in that column.

```
print(df.isnull().mean() * 100)
```

```
PassengerId      0.000000
Survived         0.000000
Pclass           0.000000
Name             0.000000
Sex              0.000000
Age             20.574163
SibSp            0.000000
Parch            0.000000
Ticket           0.000000
Fare             0.239234
Cabin           78.229665
Embarked         0.000000
dtype: float64
```

Figure 06: Percentage of Null values

In this code the percentage of missing values in each column of the dataset is determined by this code, which then outputs the results.

As a result, the output of this code will show the percentage of missing values in each column of the dataset. This can be helpful in identifying columns that might need imputation or dropping, depending on the amount and nature of the missing values.

```
#find  median value for Age
data = df
median_val = data['Age'].median()

print("Median Value of age: ",median_val)
#replacing null value with median value
data['Age'] = data['Age'].fillna(median_val)
data.isnull().sum()
```

```
Median Value of age:  27.0
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

Figure 07: Filled the missing value

This code is used to fill the missing values in the 'Age' column of the DataFrame 'data' with the median value of the column.First, the DataFrame 'data' is assigned to the DataFrame 'df'. Then, the median value of the 'Age' column is calculated using the 'median()' function and stored in the variable 'median_val'.

Next, the missing values in the 'Age' column are filled with the median value using the 'fillna()' function. Finally, the 'isnull().sum()' function is used to check if there are any missing values in the DataFrame.

```
mode_val=data['Fare'].value_counts()

data = data.fillna({"Fare": mode_val[0]})

data.isnull().sum()
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age              0
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          327
Embarked         0
dtype: int64
```

Figure 08

The `value_counts()` method, which provides a series containing counts of unique values, is being used in this code to determine the mode value for the "Fare" column in the provided dataset.

The `fillna()` method, which fills missing values with a specified value, is then used to replace the null values in the "Fare" column with the mode value. In this instance, the value_counts() output is being indexed with [0] to access the mode value.Then calls the `isnull()` function, which returns a boolean DataFrame showing which values are missing, and uses the `sum()` method to tally the amount of missing values in each column to determine whether there are still any null values in the dataset.

In the Titanic dataset, the columns that are not directly related to the target variable "Survived" are: PassengerId, Name, Ticket, and Cabin
These columns do not provide any direct information about a passenger's survival. However, some of these columns might be useful in deriving new features that could be relevant to survival prediction.

```
3] data.drop(columns=['PassengerId','Name','Ticket','Cabin'])
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	0	3	male	34.5	0	0	7.8292	Q
1	1	3	female	47.0	1	0	7.0000	S
2	0	2	male	62.0	0	0	9.6875	Q
3	0	3	male	27.0	0	0	8.6625	S
4	1	3	female	22.0	1	1	12.2875	S
...	...	...	...	...	...	...	...	...
413	0	3	male	27.0	0	0	8.0500	S
414	1	1	female	39.0	0	0	108.9000	C
415	0	3	male	38.5	0	0	7.2500	S
416	0	3	male	27.0	0	0	8.0500	S
417	0	3	male	27.0	1	1	22.3583	C

418 rows × 8 columns

Figure 9: Dropping the columns

This code is dropping the columns ‘PassengerId’, 'Name', 'Ticket', and 'Cabin' from the dataset 'data'. The function drop is used to remove columns from the DataFrame, where columns argument takes a list of column names to be removed.
Dropping these columns reduces the dimensionality of the dataset and focuses the analysis on the most important variables for predicting the 'Survived' column.

After executing this code, the updated dataset will have the columns 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', and 'Embarked’. Therefore, this is our final Titanic Dataset.